

time. Thus narrow transformations can be executed on an arbitrary subset of the data without any information about the other partitions. In contrast, transformations with wide dependencies cannot be executed on arbitrary rows and instead require the data to be partitioned in a particular way. Transformations with wide dependencies include, sort, reduceByKey, groupByKey, join, and anything that calls for repartition.

We call the process of moving the records in an RDD to accommodate a partitioning requirement, a *shuffle*. In certain instances, for example, when Spark already knows the data is partitioned in a certain way, operations with wide dependencies do not cause a shuffle. If an operation will require a shuffle to be executed, Spark adds a `ShuffledDependency` object to the dependency list associated with the RDD. In general, shuffles are expensive, and they become more expensive the more data we have, and the greater percentage of that data has to be moved to a new partition during the shuffle. As we will discuss at length in [???](#), we can get a lot of performance gains out of Spark programs by doing fewer and less expensive shuffles.

The next two diagrams illustrates the difference in the dependency graph for transformations with narrow vs. transformations with wide dependencies. On top, are narrow dependencies in which each child partition (each of the blue squares on the bottom rows) depends on a known subset of parent partitions (narrow dependencies are shown with blue arrows). The left represents a dependency graph of narrow transformations such as map, filter, mapPartitions and flatMap. On the upper right are dependencies between partitions for coalesce, a narrow transformation. In this instance we try to illustrate that the child partitions may depend on multiple parent partitions, but that so long as the set of parent partitions can be determined regardless of the values of the data in the partitions, the transformation qualifies as narrow.

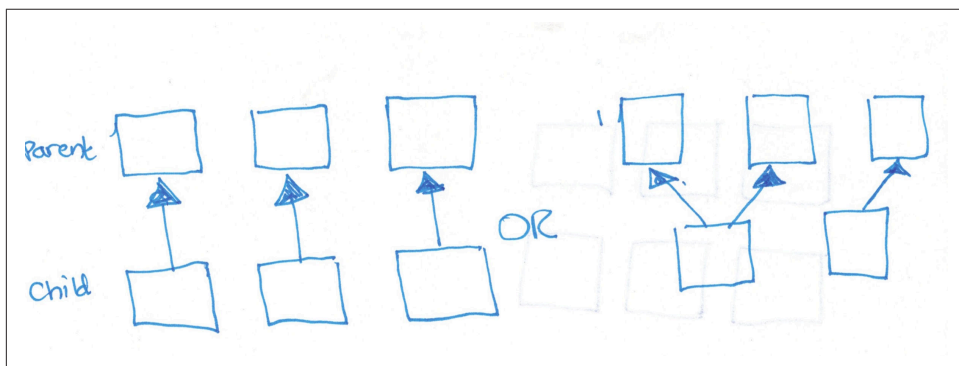


Figure 2-2. A Simple diagram of dependencies between partitions for narrow transformations.

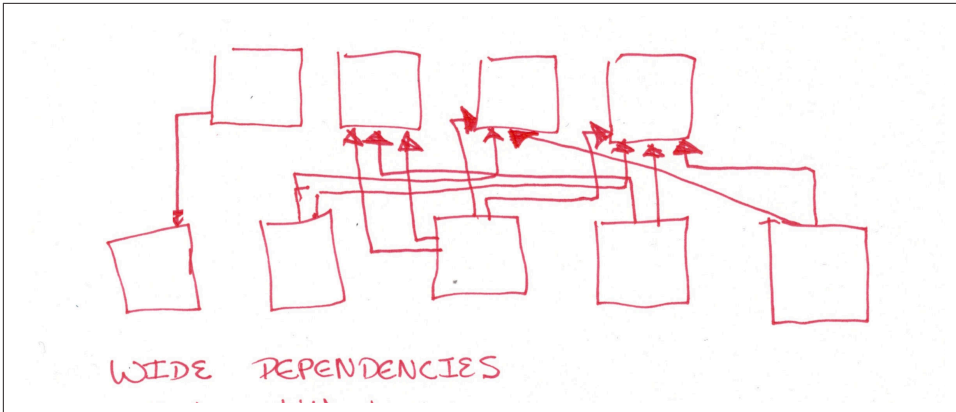


Figure 2-3. A Simple diagram of dependencies between partitions for wide transformations.

The second diagram shows wide dependencies between partitions. In this case the child partitions (shown below) depend on an arbitrary set of parent partitions. The wide dependencies (displayed as red arrows) cannot be known fully before the data is evaluated. In contrast to the *coalesce* operation, data is partitioned according to its value. The dependency graph for any operations that cause a shuffle such as *groupBy* Key, *reduceByKey*, *sort*, and *sortByKey* follows this pattern.

Join is a bit more complicated, since it can have wide or narrow dependencies depending on how the two parent RDDs are partitioned. We illustrate the dependencies in different scenarios for the join operation in “[Core Spark Joins](#)” on page 79.

Spark Job Scheduling

A Spark application consists of a driver process, which is where the high-level Spark logic is written, and a series of executor processes that can be scattered across the nodes of a cluster. The Spark program itself runs in the driver node and parts are sent to the executors. One Spark cluster can run several Spark applications concurrently. The applications are scheduled by the cluster manager and correspond to one Spark-Context. Spark applications can run multiple concurrent jobs. Jobs correspond to each action called on an RDD in a given application. In this section, we will describe the Spark application and how it launches Spark jobs, the processes that compute RDD transformations.

Resource Allocation Across Applications

Spark offers two ways of allocating resources across applications: *static allocation* and *dynamic allocation*. With static allocation, each application is allotted a finite maxi-

num of resources on the cluster and reserves them for the duration of the application (as long as the Spark Context is still running). Within the static allocation category, there are many kinds of resource allocation available, depending on the cluster. For more information, see the Spark documentation for like::[http://spark.apache.org/docs/latest/job-scheduling.html\[job scheduling\]](http://spark.apache.org/docs/latest/job-scheduling.html[job scheduling]).

Since 1.2, Spark offers the option of dynamic resource allocation which expands the functionality of static allocation. In dynamic allocation, executors are added and removed from a Spark application as needed, based on a set of heuristics for estimated resource requirement. We will discuss resource allocation in ???.

The Spark application

A Spark application corresponds to a set of Spark jobs defined by one Spark Context in the driver program. A Spark application begins when a Spark Context is started. When the Spark Context is started, each worker node starts an executor (its own Java Virtual Machine, JVM).

The Spark Context determines how many resources are allotted to each executor, and when a Spark job is launched, each executor has slots for running the tasks needed to compute an RDD. In this way, we can think of one Spark Context as one set of configuration parameters for running Spark jobs. These parameters are exposed in the SparkConf object, which is used to create a Spark Context. We will discuss how to use the parameters in ???. Often, but not always, applications correspond to users. That is, each Spark program running on your cluster likely uses one Spark Context.



RDDs cannot be shared between applications, so transformations, such as `join` that use more than one RDD must have the same Spark Context.

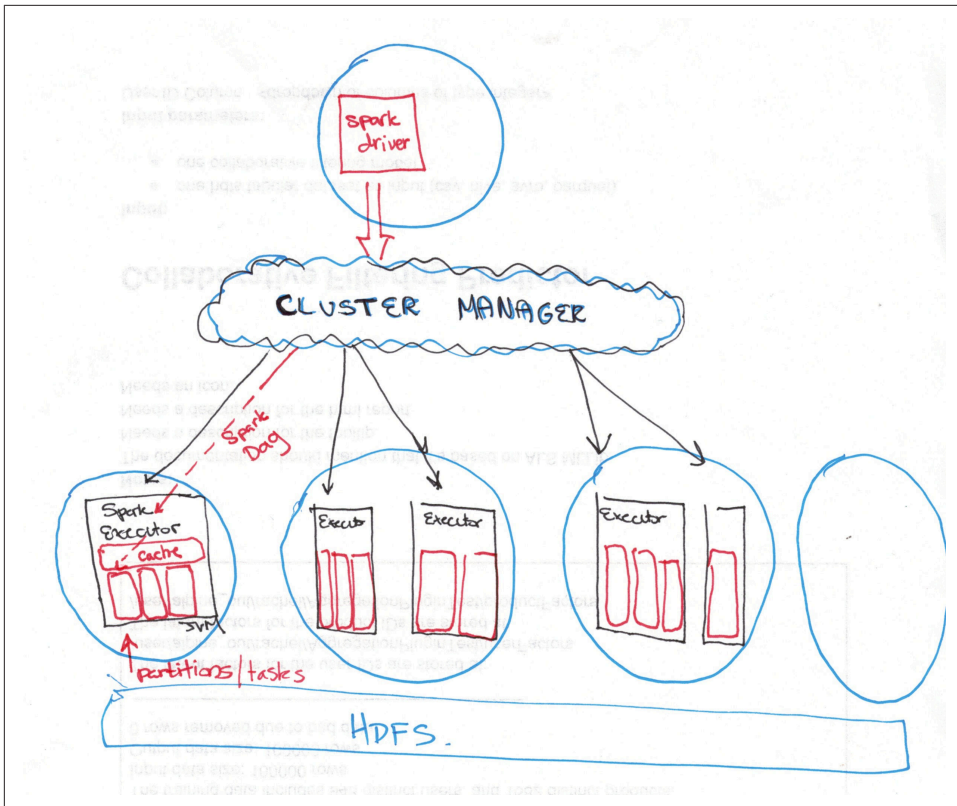


Figure 2-4. Starting a Spark application on a distributed system.

The above diagram illustrates what happens when we start a Spark Context. First The driver program pings the cluster manager. The cluster manager launches a number of Spark executors, JVMs, (shown as black boxes) on the worker nodes of the cluster (shown as blue circles). One node can have multiple Spark executors, but an executor cannot span multiple nodes. An RDD will be evaluated across the executors in partitions (shown as red rectangles). Each executor can have multiple partitions, but a partition cannot be spread across multiple executors.

By default, Spark queues jobs in a first in, first out basis. However, Spark does offer a fair scheduler, which assigns tasks to concurrent jobs in round-robin fashion, i.e. parceling out a few tasks for each job until the jobs are all complete. The fair scheduler ensures that jobs get a more even share of cluster resources. The Spark application then launches jobs in the order that their corresponding actions were called on the Spark Context.

The Anatomy of a Spark Job

In the Spark lazy evaluation paradigm, a Spark application doesn't "do anything" until the driver program calls an action. With each action, the Spark scheduler builds an execution graph and launches a *Spark job*. Each job consists of *stages*, which are steps in the transformation of the data needed to materialize the final RDD. Each stage consists of a collection of *tasks* that represent each parallel computation and are performed on the executors.

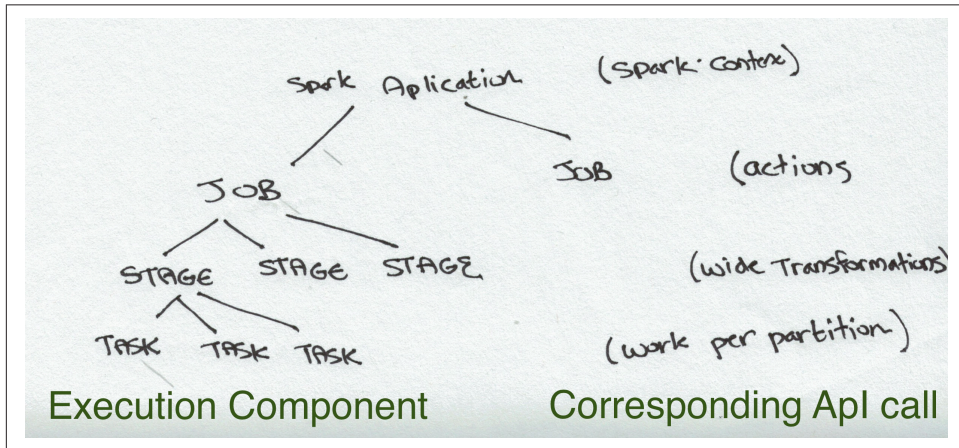


Figure 2-5. The Spark application Tree

The above diagram shows a tree of the different components of a Spark application. An application corresponds to starting a Spark Context. Each application may contain many jobs that correspond to one RDD action. Each job may contain several stages which correspond to each wide transformation. Each stage is composed of one or many tasks which correspond to a parallelizable unit of computation done in each stage. There is one task for each partition in the resulting RDD of that stage.

The DAG

Spark's high-level scheduling layer uses RDD dependencies to build a *Directed Acyclic Graph* (a DAG) of stages for each Spark job. In the Spark API, this is called the DAG Scheduler, and as you have probably noticed, errors that have to do with connecting to your cluster, your configuration parameters, or launching a Spark job show up as DAG Scheduler errors. This is because the execution of a Spark job is handled by the DAG. The DAG builds a graph of stages for each job, determines the locations to run each task, and passes that information on to the TaskScheduler, which is responsible for running tasks on the cluster.

Jobs

A Spark job is the highest element of Spark’s execution hierarchy. Each Spark job corresponds to one action, called by the Spark application. As we discussed in “[Functions on RDDs: Transformations vs. Actions](#)” on page 26, one way to conceptualize an action is as something that brings data out of the RDD world of Spark into some other storage system (usually by bringing data to the driver or some stable storage system).

Since the edges of the Spark execution graph are based on dependencies between RDD transformations (as illustrated by [Figure 2-2](#) and [Figure 2-3](#)), an operation that returns something other than an RDD cannot have any children. Thus, an arbitrarily large set of transformations may be associated with one execution graph. However, as soon as an action is called, Spark can no longer add to that graph and launches a job including those transformations that were needed to evaluate the final RDD that called the action.

Stages

Recall that Spark lazily evaluates transformations; transformations are not executed until actions are called. As mentioned above, a job is defined by calling an action. The action may include several transformations, and wide transformations define the breakdown of jobs into *stages*.

Each stage corresponds to a `ShuffleDependency` created by a wide transformation in the Spark program. At a high level, one stage can be thought of as the set of computations (tasks) that can each be computed on one executor without communication with other executors or with the driver. In other words, a new stage begins each time in the series of steps needed to compute the final RDD that data has to be moved across the network. These dependencies that create stage boundaries are called `ShuffleDependencies`, and as we discussed in “[Wide vs. Narrow Dependencies](#)” on page 26, they are caused by those wide transformations, such as `sort` or `groupByKey`, which require the data to be re-distributed across the partitions. Several transformations with narrow dependencies can be grouped into one stage. For example, a `map` and a `filter` step are combined into one stage, since neither of these transformations require a shuffle; each executor can apply the `map` and `filter` steps consecutively in one pass of the data.

Because the stage boundaries require communication with the driver, the stages associated with one job generally have to be executed in sequence rather than in parallel. It is possible to execute stages in parallel if they are used to compute different RDDs which are combined in a downstream transformation such as a `join`. However, the wide transformations needed to compute one RDD have to be computed in sequence and thus, it is usually desirable to design your program to require fewer shuffles.

Tasks

A stage consists of tasks. The *task* is the smallest unit in the execution hierarchy, and each can represent one local computation. Each of the tasks in one stage all execute the same code on a different piece of the data. One task cannot be executed on more than one executor. However, each executor has a dynamically allocated number of slots for running tasks and may run many tasks concurrently throughout its lifetime. The number of tasks per stage corresponds to the number of partitions in the output RDD of that stage.

The following diagram shows the evaluation of a Spark job that is the result of a driver program that calls the following simple Spark program:

Example 2-2.

```
def simpleSparkProgram(rdd : RDD[Double]): Long ={  
  //stage1  
  rdd.filter(_< 1000.0)  
    .map(x => (x , x) )  
  //stage2  
    .groupByKey()  
    .map{ case(value, groups) => (groups.sum, value)}  
  //stage 3  
    .sortByKey()  
    .count()  
}
```

The stages (black boxes) are bounded by the shuffle operations `groupByKey` and `sortByKey`. Each stage consists of several tasks, one for each partition in the result of the RDD transformations (shown as red squares), which are executed in parallel.

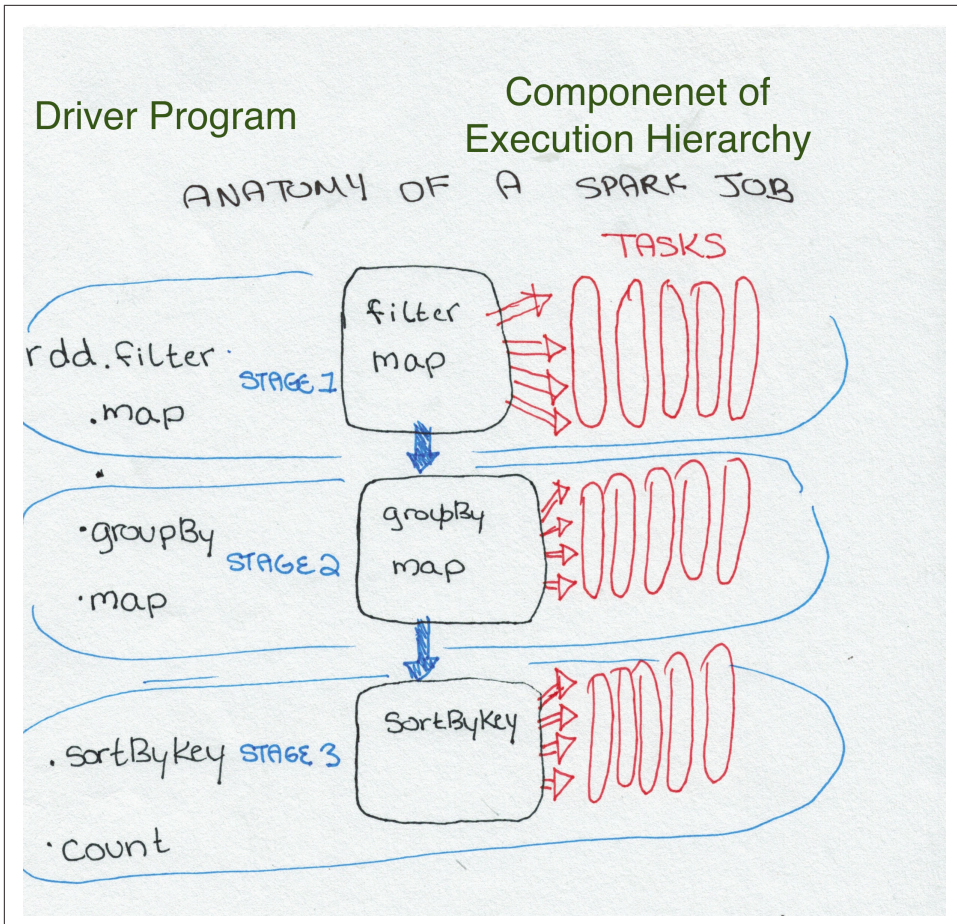


Figure 2-6. A stage diagram for the simple Spark program shown above.

In some ways, the simplest way to think of the Spark execution model is that a Spark job is the set of RDD transformations needed to compute one final result. Each stage corresponds to a segment of work, which can be accomplished without involving the driver. In other words, one stage can be computed without moving data across the partitions. Within one stage, the tasks are the units of work done for each partition of the data.

Conclusion

Spark has an innovative, efficient model of parallel computing centering on lazily evaluated, immutable, distributed datasets, RDDs. Spark exposes RDDs as an interface, and RDD methods can be used without any knowledge of their implementation.

Because of Spark's ability to run jobs concurrently, to compute jobs across multiple nodes, and to materialize RDDs lazily, the performance implications of similar logical patterns may differ widely and errors may surface from misleading places. Thus, it is important to understand how the execution model for your code is assembled in order to write and debug Spark code. Furthermore, it is often possible to accomplish the same tasks in many different ways using the Spark API, and a strong understanding of how your code is evaluated will help you optimize its performance. In this book, we will focus on ways to design Spark applications to minimize network traffic, memory errors, and the cost of failures.